

# RUTA VIVA

Documentación

Convenciones de desarrollo

Estandarización de datos

2026

Camila Marrugo

Daniel Barrera

Daniel Arciniegas



## Objetivo

Este documento establece las convenciones que debe seguir el equipo de desarrollo para garantizar que el trabajo realizado sobre el proyecto sea:

- Consistente.
- Fácil de revisar.
- Trazable.
- Auditable.
- Fácil de mantener.
- Compatible entre diferentes versiones de los datos.

Se definen dos aspectos principales:

1. **Convenciones de desarrollo:** ramas, commits y Pull Requests.
2. **Estándares de datos:** nombres, formatos y unificación de las columnas de los archivos.

## Glosario de dominio

El presente glosario define los principales términos utilizados en el proyecto RUTA VIVA, con el objetivo de establecer un lenguaje común entre los integrantes del equipo y evitar ambigüedades durante el desarrollo, transformación y análisis de los datos.

| Término          | Definición                                                                                                                                                                     |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RUTA VIVA</b> | Nombre del proyecto orientado al procesamiento, estandarización y organización de información proveniente de diferentes fuentes y versiones de datos.                          |
| <b>F1</b>        | Archivo o fuente de datos principal utilizada en el proyecto. Puede presentar diferencias en nombres, formatos y estructura dependiendo del año o de la fuente de información. |



|                                 |                                                                                                                                                                    |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Archivo histórico</b>        | Archivo correspondiente a una versión o periodo anterior de los datos, que debe ser transformado al esquema estándar antes de integrarse al proceso.               |
| <b>Esquema</b>                  | Estructura definida para los datos, incluyendo las columnas, sus nombres, tipos de datos y reglas que deben cumplir.                                               |
| <b>Esquema estándar</b>         | Estructura única utilizada como referencia para que los datos provenientes de diferentes años o fuentes puedan integrarse de manera consistente.                   |
| <b>Estandarización de datos</b> | Proceso mediante el cual los datos de diferentes fuentes o versiones se transforman para cumplir con un mismo conjunto de nombres, formatos, tipos y reglas.       |
| <b>Normalización</b>            | Proceso de transformación de valores o nombres diferentes hacia una representación común y consistente.                                                            |
| <b>Columna obligatoria</b>      | Campo que debe estar presente en el esquema final después de realizar la transformación de los datos.                                                              |
| <b>Identificador</b>            | Valor utilizado para distinguir de manera única un registro o entidad. Los identificadores deben manejarse como texto para conservar valores como ceros iniciales. |
| <b>Valor nulo (NULL)</b>        | Representación utilizada para indicar que un dato está ausente, no disponible o no pudo ser obtenido durante el proceso de transformación.                         |



|                            |                                                                                                                                              |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Transformación</b>      | Operación aplicada sobre los datos para modificar su estructura, formato, nombre o representación y adaptarlos al esquema estándar.          |
| <b>Pipeline</b>            | Conjunto de etapas o procesos por los que pasan los datos desde su fuente de origen hasta las etapas posteriores de procesamiento.           |
| <b>Trazabilidad</b>        | Capacidad de identificar el origen, procesamiento, versión y resultado de los datos durante una transformación.                              |
| <b>Control de calidad</b>  | Conjunto de validaciones realizadas después de la transformación para comprobar que los datos cumplen con las reglas y estructura esperadas. |
| <b>Versión del esquema</b> | Identificador que permite distinguir diferentes versiones de la estructura de datos y conocer qué cambios se realizaron entre ellas.         |
| <b>Dato de origen</b>      | Información tal como se encuentra en el archivo o fuente original antes de aplicar las transformaciones.                                     |
| <b>Dato estandarizado</b>  | Información que ya fue transformada para cumplir con las convenciones y reglas definidas por el proyecto.                                    |

## Propósito del glosario

El glosario debe utilizarse como referencia durante el desarrollo, documentación y mantenimiento del proyecto. Los términos definidos en esta sección deberán conservar un significado consistente en los procesos de transformación, documentación técnica y comunicación del equipo.



# 1. Convenciones de desarrollo

## 1.1 Convención de ramificaciones

Se utilizará Git Flow simplificado para organizar el trabajo del equipo.

Las ramas principales serán:

| Rama       | Propósito                                                    |
|------------|--------------------------------------------------------------|
| main       | Contiene las versiones estables del proyecto.                |
| develop    | Contiene los cambios que están en desarrollo.                |
| feature/*  | Desarrollo de nuevas funcionalidades.                        |
| fix/*      | Corrección de errores.                                       |
| docs/*     | Cambios exclusivamente relacionados con documentación.       |
| refactor/* | Cambios de estructura o código que no agregan funcionalidad. |

### Ejemplos

feature/normalizacion-f1  
feature/carga-postgresql  
fix/error-fecha-f1  
docs/data-standards  
refactor/limpieza-pipeline

## Reglas para las ramas

- Los nombres deben escribirse en minúsculas.
- Las palabras deben separarse mediante -.
- No se deben utilizar espacios.
- El nombre debe describir claramente el objetivo de la rama.
- Una rama debe estar relacionada con la tarea específica (Una corrección no debe estar en docs a menos que sea documentación).
- No se deben mezclar funcionalidades diferentes dentro de una misma rama.



### Ejemplo correcto

feature/normalizacion-f1

### Ejemplo incorrecto

MiNuevaRama

## 2. Convención de commits

Los commits utilizarán una estructura basada en **Conventional Commits**.

La estructura será:

tipo: descripción breve de lo que se realizó o se debe realizar.

### 2.1 Tipos permitidos

| Tipo     | Uso                                                          |
|----------|--------------------------------------------------------------|
| feat     | Nueva funcionalidad.                                         |
| fix      | Corrección de errores.                                       |
| docs     | Cambios en documentación.                                    |
| refactor | Reestructuración del código sin cambiar su funcionalidad.    |
| test     | Creación o modificación de pruebas.                          |
| data     | Cambios relacionados con datos, esquemas o transformaciones. |
|          |                                                              |

### Ejemplos

feat: agregado transformación de archivos F1

fix: corregido conversión de fechas F1

docs: documentación creada estándares de datos

data: normalizado nombres de columnas F1

test: agregado pruebas para transformación F1

### 2.2 Reglas de los commits



Los commits deben:

- Describir claramente qué cambio se realizó.
- La descripción debe ser lo más pequeño posible y relacionada con una única modificación.
- Utilizar verbos en infinitivo o una descripción consistente.
- Evitar mensajes genéricos como:

cambios

actualización

arreglo

En su lugar:

fix: corregido formato de fecha en archivo F1

## 3. Pull Requests

Todo cambio que llegue a `main` o `develop` debe realizarse mediante un Pull Request.

### 3.1 Requisitos del Pull Request

El Pull Request debe incluir:

- Descripción del cambio.
- Objetivo de la modificación.
- Archivos afectados.
- Evidencia de pruebas realizadas cuando corresponda.
- Referencia a la tarea o issue relacionada.

#### Ejemplo

##### Descripción

Se implementó la normalización de los archivos F1 para unificar los nombres y formatos de las columnas.

##### Cambios realizados

- Unificación de nombres de columnas.
- Normalización de fechas.
- Conversión de valores numéricos.
- Documentación del esquema resultante.

##### Pruebas



- Se verificó que las columnas tengan el nombre esperado.
- Se verificó el formato de las fechas.
- Se verificó que no existan columnas duplicadas.

## 3.2 Revisión

Antes de realizar el merge:

- El código debe ser revisado por al menos otro integrante del equipo cuando sea posible.
- Las pruebas correspondientes deben ejecutarse correctamente.
- Los conflictos deben resolverse antes del merge.
- La documentación debe actualizarse si el cambio modifica el comportamiento o los datos.

# 4. Estándares de datos

## 4.1 Objetivo

Los archivos F1 pueden presentar cambios en su estructura dependiendo del año o de la fuente de información.

Por esta razón, antes de utilizar los datos en procesos posteriores se debe aplicar una estandarización que permita trabajar con un esquema común.

El objetivo es que diferentes versiones de los archivos F1 puedan transformarse a una estructura única.

# 5. Convención para nombres de columnas

Todas las columnas deberán utilizar:

- Letras minúsculas.
- Palabras separadas mediante \_.
- Sin espacios.
- Sin tildes.
- Sin caracteres especiales.
- Nombres descriptivos.

## Ejemplo

Si un archivo contiene:



Nombre Completo  
 Fecha Nacimiento  
 Tipo Documento  
 Número Documento

El esquema estandarizado será:

nombre\_completo  
 fecha\_nacimiento  
 tipo\_documento  
 numero\_documento

## 5.1 Ejemplos de normalización

| Nombre original      | Nombre estandarizado |
|----------------------|----------------------|
| Nombre               | nombre               |
| NOMBRE               | nombre               |
| Nombre Completo      | nombre_completo      |
| Fecha Nacimiento     | fecha_nacimiento     |
| Fecha_Nacimient<br>o | fecha_nacimiento     |
| N° Documento         | numero_documento     |
| Número Documento     | numero_documento     |

## 6. Unificación de columnas entre años

Cuando una columna tenga nombres diferentes dependiendo del año, se debe establecer un único nombre estándar.

Por ejemplo:



| Año  | Nombre original |
|------|-----------------|
| 2022 | ID_PERSONA      |
| 2023 | IDENTIFICACION  |
| 2024 | NUM_DOCUMENTO   |

Las tres columnas pueden representar el mismo concepto.

El nombre estándar será:

numero\_documento

Por lo tanto, durante la transformación se deberá realizar la equivalencia:

ID\_PERSONA  
 IDENTIFICACION  
 NUM\_DOCUMENTO  
 ↓  
 numero\_documento

Esto permite que los datos de diferentes años puedan integrarse sin generar columnas duplicadas.

## 7. Tipos de datos

Además del nombre de las columnas, se debe establecer un tipo de dato estándar.

### Tipo de información    Tipo estándar

|                    |          |
|--------------------|----------|
| Identificadores    | string   |
| Nombres            | string   |
| Texto              | string   |
| Fechas             | date     |
| Fecha y hora       | datetime |
| Cantidades enteras | integer  |
| Valores decimales  | decimal  |
| Valores booleanos  | boolean  |



## 7.1 Identificadores

Los identificadores(ID, cédula) deben tratarse como texto aunque contengan únicamente números.

Por ejemplo:

00123456

No debe convertirse automáticamente a:

123456

porque se perderían los ceros iniciales.

Por lo tanto:

numero\_documento → string

## 8. Estandarización de fechas

Todas las fechas deberán utilizar un formato único:

YYYY-MM-DD

Ejemplo:

2026-09-25

Si el archivo original contiene:

25/09/2026

o:

09-25-2026

deberá transformarse al formato estándar:

2026-09-25

Las fechas inválidas o que no puedan ser interpretadas correctamente deberán identificarse para su posterior revisión.



## 9. Estandarización de valores de texto

Los valores de texto deben seguir una representación consistente.

Por ejemplo, si una columna representa el sexo y diferentes archivos contienen:

M  
 Masculino  
 male  
 Hombre

se deberá definir una representación estándar:

M  
 F

La transformación debe estar documentada.

### Ejemplo

| Valor original | Valor estándar |
|----------------|----------------|
| M              | M              |
| Masculino      | M              |
| Male           | M              |
| Hombre         | M              |
| F              | F              |
| Femenino       | F              |
| Female         | F              |
| Mujer          | F              |

## 10. Valores nulos

Los valores que representen información faltante deberán convertirse a un valor nulo estándar.



Ejemplos de valores que deben identificarse:

N/A

NA

NULL

null

-

Sin información

No disponible

Después de la transformación deberán representarse como:

NULL

o como el equivalente utilizado por la herramienta de procesamiento.

No se deben interpretar automáticamente los valores faltantes como 0, ya que 0 representa un valor real diferente a la ausencia de información.

## 11. Columnas obligatorias

El esquema final de F1 deberá definir cuáles columnas son obligatorias.

Ejemplo:

id

fecha

numero\_documento

nombre

Estas columnas deben estar presentes después de la transformación.

Si una versión anual del archivo no contiene una columna obligatoria, el proceso debe:

1. Detectar la ausencia.
2. Registrar la incidencia.
3. Determinar si la columna puede obtenerse de otra fuente.
4. Si no es posible obtenerla, establecer el valor como nulo y registrar la situación.

## 12. Control de calidad

Después de estandarizar un archivo F1 se deben realizar validaciones.



## Validaciones mínimas

- Verificar que existan las columnas esperadas.
- Verificar que no existan columnas duplicadas.
- Verificar los tipos de datos.
- Verificar los formatos de fecha.
- Identificar valores nulos.
- Identificar valores fuera de los valores permitidos.
- Verificar duplicados en identificadores cuando corresponda.
- Comparar la cantidad de registros antes y después de la transformación.

## 13. Esquema

El resultado de la transformación debe utilizar un esquema.

Ejemplo:

```
f_nombre
├── id
├── numero_documento
├── nombre
├── apellido
├── fecha_nacimiento
├── genero
├── ciudad
├── departamento
└── fecha_carga
```

Este esquema representa la estructura estándar que deben utilizar los procesos posteriores.

Los nombres originales de los archivos históricos no deben pasar directamente a las siguientes etapas del pipeline.

## 14. Trazabilidad

Cada transformación debe poder ser auditada.

Se debe conservar información que permita identificar:

- Archivo de origen.
- Año del archivo.
- Fecha de procesamiento.



- Nombre de la transformación.
- Versión del esquema.
- Cantidad de registros de entrada.
- Cantidad de registros de salida.
- Errores encontrados.

Ejemplo:

archivo\_origen: F1\_2026.csv

año: 2026

schema\_version: 1.0

fecha\_procesamiento: 2026-10-25

registros\_entrada: 10000

registros\_salida: 998

## 15. Versionamiento del esquema

Los cambios importantes en el esquema deben versionarse.

Ejemplo:

v1.0

Primera versión del esquema.

v1.1

Se agrega una columna sin modificar las existentes.

v2.0

Se modifica o elimina una columna existente o cambia su significado.

Esto permite conocer qué estructura utilizaba cada proceso y facilita la auditoría histórica.